

```

def min_vertex_cover_version1(self):
    """
    Approximation algorithm.
    Returns: (size of cover, set of vertices in the cover)
    """
    edges = set()
    for u in self._out_edges:
        for v in self._out_edges[u]:
            edges.add((u, v))
    vertices = set()
    while edges:
        u, v = edges.pop() #select an edge
        vertices.add(u) #add the two of them to the solution
        vertices.add(v)
        #remove all edges incident to u or v
        new_edges = set()
        for e in edges:
            if u not in e and v not in e:
                new_edges.add(e)
        edges = new_edges
    return len(vertices), vertices

def min_vertex_cover_version2(self):
    """
    Greedy heuristic.
    Returns: (size of cover, set of vertices in the cover)
    """
    edges = set()
    for x in self._out_edges:
        for y in self._out_edges[x]:
            edges.add((x, y))
    vertices = set()
    while edges:
        #for all the edges left count how many times each vertex appears
        degree_count = {}
        for u, v in edges:
            degree_count[u] = degree_count.get(u, 0) + 1
            degree_count[v] = degree_count.get(v, 0) + 1
        #pick the one with the maximum degree
        best_vertex = max(degree_count, key=degree_count.get)
        #add it to the solution
        vertices.add(best_vertex)
        #remove all edges incident to best_vertex
        edges = {(u, v) for (u, v) in edges if u != best_vertex and v !=
best_vertex}
    return len(vertices), vertices

```